AI 388 / CS 388 Natural Language Processing

Module 1: Linear Classification

Topics 1, 2, 3, 4, 7

Fall 2026 midterm study guide

Built from the lecture videos/transcripts, slides, Canvas exercises, Assignments 1–3 and the Fall 2022/2023 midterms.

Prepared October 2026 · lecture notation throughout

How to use this guide

Each **Topic** number is the item number on the official Fall 2026 midterm topic list. Section numbers are *topic.section*, so "2.4" is Topic 2, Section 4. Every topic has the same layout:

- **In plain words**: the idea in one paragraph.
- **Key boxes**: definitions and formulas in **lecture notation**.
- **Steps**: the algorithm or exam procedure, step by step.
- **Worked examples**: full calculations (checked numerically with Python).
- **Past exam**: every relevant Fall 2022/2023 midterm question and Canvas exercise, with the official answer and an explanation. Official errata are noted.
- **Traps**: places where convention differences or common slips cost points.
- **Practice**: original questions, with answers right after.

Notation (lecture): input $\mathbf{x}$; feature vector $f(\mathbf{x}) \in \mathbb{R}^d$; weights $\mathbf{w}$; label $y \in \{-1, +1\}$ (binary) or $y \in \mathcal{Y}$ (multiclass); step size α; D training examples $(\mathbf{x}^{(i)}, y^{(i)})$; log is the natural log unless a question says otherwise. Perceptron decision rule: positive iff $\mathbf{w}^\top f(\mathbf{x}) > 0$ (a score of 0 is negative). Where a past exam used a different convention, the box says so in red.

Exam reminders. Auto-graded short answers must match the requested format exactly (rounding, fractions vs. decimals, list syntax like `[w1, w2]`). Adding explanation text makes the answer wrong. Long answers: show work concisely for partial credit and box final answers.

Contents

How to use this guide 1

1 Bag-of-Words Features 4

1.1 Setup and notation (lecture) . . . 4
1.2 The feature-extraction pipeline . . . 4
1.3 Geometry: what the feature space looks like . . . 5
1.4 Which preprocessing shrinks the feature space? . . . 6
1.5 Exercise 1.0 (Canvas) . . . 7
1.6 Practice . . . 7

2 Binary Perceptron: Algorithm and Loss 8

2.1 Algorithm (lecture form) . . . 8
2.2 Worked trace: the lecture example . . . 8
2.3 Non-separable data and the fix . . . 9
2.4 Perceptron as SGD on a loss . . . 10
2.5 Past exam problems . . . 10
2.6 Practice . . . 11

3 Binary Logistic Regression 13

3.1 Model . . . 13
3.2 Training objective . . . 13
3.3 Gradient derivation (be able to reproduce) . . . 14
3.4 Worked numeric example . . . 14
3.5 Past exam / exercise problems . . . 15
3.6 Practice . . . 16

4 Sentiment Analysis: Useful Features 17

4.1 What makes sentiment hard (lecture examples) . . . 17
4.2 Useful features (what the exam can ask) . . . 17
4.3 History (lecture 1.4.0) — MC fodder . . . 18
4.4 A1 connection . . . 18
4.5 Past exam problems . . . 18
4.6 Practice . . . 19

7 Optimization: SGD, Step Size, Initialization 20

7.1 The optimization problem 20
7.2 Step size: the quadratic example 20
7.3 Choosing step sizes (lecture recipe) 21
7.4 Regularization (lecture: "don't really use") 22
7.5 Initialization (neural networks) 22
7.6 Practice 23

TOPIC 1

Bag-of-Words Features

Sources: Lectures 1.1.0 Linear Binary Classification, 1.1.1 Sentiment Analysis & Basic Feature Extraction, 1.4.0 Sentiment Analysis; slides seg-1, seg-2, seg-7; Exercise 1.0; Assignment 1; Fall 2022 Q1(4); Fall 2023 Q2(a), Q1(15).

In plain words

A classifier only understands numbers, but our input is a string. A *feature extractor* f turns the string into a long vector. In **bag-of-words** (BoW), the vector has one slot per vocabulary word. A slot holds how many times that word appears (or just 1/0 for present/absent). Word order is thrown away: the sentence becomes a "bag" of words. Each vocabulary word is one *axis* of the feature space, so a sentence is a point in $\mathbb{R}^{|V|}$.

1.1 Setup and notation (lecture)

- Input $\mathbf{x}$ (usually a string). Feature extractor f: string $\to f(\mathbf{x}) \in \mathbb{R}^d$.
- Label $y \in \{-1, +1\}$ for binary classification (negative/positive).
- Classifier = weight vector $\mathbf{w} \in \mathbb{R}^d$ (some texts write θ). Same dimension d as the features.
- Decision: positive iff $\mathbf{w}^\top f(\mathbf{x}) > 0$. The decision boundary $\{f(\mathbf{x}) : \mathbf{w}^\top f(\mathbf{x}) = 0\}$ is perpendicular to $\mathbf{w}$.
- **No separate bias.** Instead of $\mathbf{w}^\top f(\mathbf{x}) + b$, append a constant feature 1: $f(\mathbf{x}) = [3, -1, 2] \mapsto [3, -1, 2, 1]$. The last weight then plays the role of b. (The lecture: "we are going to not use [b].")

1.2 The feature-extraction pipeline

Raw string → feature vector (lecture 1.1.1, five steps)

1. **Tokenization.** Split the string into tokens. Whitespace splitting leaves `great!` and `great` as different words. A tokenizer separates punctuation and contractions: `great!`→`great !`, `wasn't`→`was n't`.
2. **[Sometimes] Stop-word removal.** Drop function words (*the, of, a*). These skew the vectors and carry little sentiment.
3. **[Sometimes] Casing.** Lowercase everything, or *truecase* (e.g. for speech-recognizer output that has no case).
4. **Unknown words.** Words outside the vocabulary (e.g. the 10,000 most frequent) become `UNK`, or are simply dropped for sentiment.
5. **Indexing.** Map each word or n-gram to an integer position in the vector, using a map (A1's `Indexer`): `the`→5, `great`→247, ...

1.2.1 Count vs. presence features

- **Counts:** the value is how many times the word occurs ("the" twice ⇒ 2).
- **Presence/absence (indicator):** the value is 1 if the word occurs at all, else 0.

For "the movie was great" with a 10,000-word vocabulary there are **four 1s and 9,996 zeros**, so the vector is extremely *sparse*. In code you store only the nonzero entries (A1 suggests a `Counter` mapping index→value). The weight vector itself is a dense numpy array.

1.2.2 Bag-of-n-grams

An n-gram is a sequence of n consecutive words. The 2-grams (bigrams) of "the movie was great" are *the movie, movie was, was great*. A sentence with L tokens has $L - n + 1$ n-grams (more if you add start/end tokens). Bigrams can capture phrases like *not great* and *watch again*, which unigrams cannot. The cost is a much larger feature space (e.g. about 100,000 bigrams instead of 10,000 words). A1 tip: encode mixed feature types with "magic strings" in one indexer, e.g. `Unigram=great` and `Bigram=great|movie`.

1.2.3 TF-IDF (introduced, not used much)

TF-IDF

$$\begin{aligned} \text{tf}(w,d) &= \text{count of } w \text{ in document } d \\ \text{idf}(w) &= \log \frac{\text{total \# documents}}{\text{\# documents containing } w} \\ \text{tf-idf}(w,d) &= \text{tf}(w,d) \times \text{idf}(w) \end{aligned}$$

A common word (*the*) appears in almost every document, so idf $\approx \log 1 = 0$ and its value shrinks toward 0. A rare word like *mango* appearing in 1 of 10,000 documents gets $\log_{10}(10000/1) = 4$ (the lecture's example uses base 10). The effect: emphasize words characteristic of this document.

1.3 Geometry: what the feature space looks like

Each feature is an axis. With vocabulary [good, bad, not], "not bad" $\mapsto (0,1,1)$ is a corner of the unit cube. A linear classifier is a plane through the origin (no bias feature) separating the $+$ points from the $-$ points. Changing the feature set changes the geometry. Adding bigram axes (*not good, not bad*) can make inseparable data separable (Topic 2, the good/bad/not good/not bad example).

Worked example: building vectors

Vocabulary, in this order: [a, bad, good, great, movie, not, the, was]. Lowercase and tokenize; drop words not in the vocabulary.
Sentence: *The movie was not bad, the acting was great!* Tokens: the movie was not bad , the acting was great ! In-vocabulary tokens: the(2) movie(1) was(2) not(1) bad(1) great(1). "acting", "," and "!" are dropped.

$$\text{count } f(\mathbf{x}) = [\,0,1,0,1,1,1,2,2\,], \qquad \text{presence } f(\mathbf{x}) = [\,0,1,0,1,1,1,1,1\,].$$

Bigrams of *the movie was not bad* (5 tokens $\Rightarrow$ 4 bigrams): the movie, movie was, was not, **not bad**. With a bias feature appended, the count vector becomes [0,1,0,1,1,1,2,2,1].

Worked example: TF-IDF

Four documents: d_1 "the movie was great", d_2 "the movie was bad", d_3 "the plot was great great", d_4 "a great cast".

word	docs containing	idf $= \log_{10}(4/\text{df})$	idf with ln
the	3	$\log_{10}(4/3) = 0.1249$	0.2877
great	3	0.1249	0.2877
bad	1	$\log_{10} 4 = 0.6021$	1.3863

tf-idf of *great* in $d_3 = 2 \times 0.1249 = 0.2499$ (base 10). *bad* in d_2 gets 1×0.6021, more than twice as large, even though it occurs once. If a question doesn't state the log base, say which one you used.

1.4 Which preprocessing shrinks the feature space?

Midterm Fall 2022 Q1(4)

Unigram BoW model. Which preprocessing techniques reduce the size of the feature space? I. lowercasing; II. bigrams instead of unigrams; III. stemming; IV. removing stopwords; V. tokenizing punctuation (`great!`→`great !`); VI. tf-idf weighting.

Official answer: I, III, IV, V.

Why: I merges *Great*/*great*. III merges *loved*/*love*. IV deletes features. V removes `great!`, `great.`, `great,...` as separate features, adding only a few punctuation tokens. II *increases* the space. VI changes the *values* in the vector, not the number of dimensions.

Midterm Fall 2023 Q2(a)

Vocabulary [good, great, not]. Examples: `goodx` +; `great` +; `not greatq` −; `not good` − (typos are deliberate). Write the feature vectors.

Official answer: [0,0,0], [0,1,0], [0,0,1], [1,0,1].

Why: `goodx`≠`good` and `greatq`≠`great`: an exact string match fails, so a typo makes the word vanish. The first example becomes the all-zero vector. (This motivates subword tokenization, Topic 17, and A2's typo setting.)

Midterm Fall 2023 Q1(15) (also see Topic 17)

Train a sentence classifier on data with only singular nouns, then test on data with plural nouns. Which could help generalization? I. stemming; II. subword tokenization; III. pre-trained word embeddings; IV. BoW featurization; V. running a syntactic parser.

Official answer: I, II, III.

Why: Stemming maps *dogs*→*dog*. Subwords give *dog* + *s*, sharing the *dog* piece. Pre-trained embeddings put *dogs* near *dog*. BoW alone treats *dogs* as an unseen feature. A parser doesn't obviously help.

1.5 Exercise 1.0 (Canvas)

Canvas Exercise 1.0 Q1–Q2

(1) "the movie was good" + / "the movie was bad" −. Which two features are most helpful?

Answer: Indicator(contains "good"), Indicator(contains "bad"). Either one alone is enough.

(2) "the movie was good" + / "not good" − / "not not good" +. Which further feature helps?

Answer: Indicator(Count("not") is odd). The label flips with each *not*. A plain Count("not") is monotone in the number of *not*s, so a linear model can't use it to make the label alternate.

Traps

- **Follow the stated vocabulary order** when writing vectors. Auto-graded answers compare the exact list.
- **Out-of-vocabulary words contribute nothing** (all-zero vector possible). A typo makes the word out of vocabulary.
- Count vs. indicator: read the question. "Bag-of-words" in lecture defaults to counts; A1 code (and many exam questions) use 0/1.
- Bias feature: include it only if the question says so. If you include it, it's usually the last coordinate, always 1.
- BoW is *order-blind*: "good, not bad" and "bad, not good" get identical unigram vectors.
- Number of n-grams in L tokens is $L - n + 1$ (no padding tokens).

1.6 Practice

Practice

1. Vocabulary [bad, boring, fun, not, very]. Give count vectors for (a) "very very fun" (b) "not boring , not bad" (c) "fantastic".
2. How many bigrams does a 12-token sentence have? How many trigrams?
3. 1,000 documents. "excellent" appears in 10 of them; in document d it occurs 3 times. Compute tf-idf with $\log_{10}$.
4. True/False: applying tf-idf weighting reduces the dimensionality of the BoW feature space.
5. Give two sentences with identical unigram BoW vectors but opposite sentiment. Which feature type fixes this?

Answers

1. (a) [0,0,1,0,2] (b) [1,1,0,2,0] (c) [0,0,0,0,0]. 2. 11 bigrams, 10 trigrams. 3. $3 \times \log_{10}(1000/10) = 3 \times 2 = 6$. 4. False: same dimensions, different values. 5. e.g. "it was good , not bad" vs. "it was bad , not good". Bigram features (*not bad* vs. *not good*) separate them.

TOPIC 2

Binary Perceptron: Algorithm and Loss

Sources: Lectures 1.2.0 Perceptron, 1.2.1 Perceptron as Minimizing Loss, 1.1.2 Basics of Learning, Gradient Descent; slides seg-3, seg-4, seg-5; Exercise 1.1; A1 Part 1; Fall 2022 Q3; Fall 2023 Q1(1), Q2(b).

In plain words

Go through the examples one at a time. If the current weights classify an example correctly, do nothing. If they get a positive example wrong, **add** its feature vector to $\mathbf{w}$, which pushes its score up next time. If they get a negative example wrong, **subtract** its feature vector. Repeat for several passes (epochs). If some linear boundary separates the data, this provably stops making mistakes. If not, it cycles forever.

2.1 Algorithm (lecture form)

Perceptron

Decision rule (lecture): $y_{\text{pred}} = +1$ if $\mathbf{w}^\top f(\mathbf{x}) > 0$, else -1. A score of exactly 0 is **negative**.
Initialize $\mathbf{w} = \mathbf{0}$.
for $t = 1 \dots$ num_epochs:
  for $i = 1 \dots D$:
    $y_{\text{pred}} \leftarrow \text{decision}(\mathbf{w}^\top f(\mathbf{x}^{(i)}))$
    if $y_{\text{pred}} = y^{(i)}$: no change
    else if $y^{(i)} = +1$: $\mathbf{w} \leftarrow \mathbf{w} + \alpha\, f(\mathbf{x}^{(i)})$
    else $(y^{(i)} = -1)$: $\mathbf{w} \leftarrow \mathbf{w} - \alpha\, f(\mathbf{x}^{(i)})$
Compact form of the update on a mistake: $\boxed{\mathbf{w} \leftarrow \mathbf{w} + \alpha\, y^{(i)}\, f(\mathbf{x}^{(i)})}$.

- $\mathbf{w}$ and $f(\mathbf{x})$ live in the same d-dimensional space, so they can be added.
- **Step size α doesn't matter if $\mathbf{w}$ starts at 0 and α is constant.** Every update is a multiple of α, so $\mathbf{w} = \alpha \cdot (\text{integer combination of features})$. The sign of every score, and hence every decision, is unchanged. Exams often say "assume $\alpha = 1$". (This is a consequence of the algorithm, not a lecture statement. It fails if $\mathbf{w}$ starts nonzero.)
- In practice: shuffle the data each epoch (A1 handout) and run a fixed number of epochs.
- **A1 code form** uses labels $y \in \{0, 1\}$ and predictions $\hat{y} \in \{0, 1\}$: `w += lr * (label - prediction) * value`. Here $(\text{label} - \hat{y}) \in \{-1, 0, +1\}$, so this is exactly the same algorithm.

2.2 Worked trace: the lecture example

Worked example: lecture 1.2.0 (features: Movie, Good, Bad, Not; $\alpha = 1$; rule > 0)

Data: "movie good" $+1$ [1,1,0,0]; "movie bad" -1 [1,0,1,0]; "not good" -1 [0,1,0,1].

epoch	example	y	score $\mathbf{w}^\top f(\mathbf{x})$	pred	$\mathbf{w}$ after
1	movie good	$+1$	0	-1 wrong	[1,1,0,0]
1	movie bad	-1	1	$+1$ wrong	[0,1,-1,0]
1	not good	-1	1	$+1$ wrong	[0,0,-1,-1]
2	movie good	$+1$	0	-1 wrong	[1,1,-1,-1]
2	movie bad	-1	0	-1 ok	[1,1,-1,-1]
2	not good	-1	0	-1 ok	[1,1,-1,-1]
3	all three		2, 0, 0	all ok	**converged**

Learned: *bad* and *not* negative, *movie* and *good* positive. Two examples sit exactly at score 0 and are classified negative only because of the tie rule.

How to trace a perceptron quickly (exam procedure)

1. Write every $f(\mathbf{x})$ as a row in the given feature order. Note the tie rule (> 0 or ≥ 0) and α.
2. Keep a table: score $\rightarrow$ prediction $\rightarrow$ correct? $\rightarrow$ new $\mathbf{w}$.
3. On a mistake add $\alpha f(\mathbf{x})$ ($y = +1$) or subtract it ($y = -1$). Otherwise copy $\mathbf{w}$ unchanged.
4. "One epoch" means one pass in the given order. "Until convergence" means repeat until a full pass has no mistakes.
5. Sanity check: the final $\mathbf{w}$ is always $\alpha \sum(\pm\text{features of mistaken examples})$.

2.3 Non-separable data and the fix

Worked example: lecture's XOR-like example

Features [good, bad, not]: "good" + [1,0,0]; "bad" − [0,1,0]; "not good" − [1,0,1]; "not bad" + [0,1,1].
Suppose some $\mathbf{w}$ classified all four correctly (with the > 0 rule): $w_g > 0$, $w_b \leq 0$, $w_g + w_n \leq 0$, $w_b + w_n > 0$. The last two give $w_n \leq -w_g < 0$ and $w_n > -w_b \geq 0$. That's a contradiction, so the data are **not linearly separable** and the perceptron **loops forever**.
One epoch from $\mathbf{w} = 0$ (Exercise 1.1 Q2): [1,0,0]→[1,0,0]→[0,0,-1]→[0,1,0]. The final answer is **[0,1,0]**, and further epochs keep cycling.
Fix with features: add bigram features *not good*, *not bad*. In 5-D the data become separable, e.g. $\mathbf{w}$ with $w_{\text{not bad}}$ large positive and $w_{\text{not good}}$ large negative. **Fix with model:** a nonlinear classifier (feed-forward NN, Topic 8). Switching to logistic regression or adding epochs does *not* help (Exercise 1.1 Q3).

Convergence facts

- If the data are linearly separable, the perceptron converges (finds a separating $\mathbf{w}$) in finitely many updates.
- If not separable, it never converges. Weights cycle and accuracy fluctuates.
- It is **not** always guaranteed to converge (Fall 2023 Q1(1) statement I is false).
- With many features (e.g. big BoW vocabularies) data are often separable in practice. Feature design is what changes separability.

2.4 Perceptron as SGD on a loss

Perceptron loss (lecture 1.2.1)

$$\mathcal{L}(\mathbf{x}^{(i)}, y^{(i)}, \mathbf{w}) = \begin{cases} 0 & \text{if the prediction is correct} \\ -y^{(i)}\,\mathbf{w}^\top f(\mathbf{x}^{(i)}) & \text{otherwise} \end{cases} \qquad \text{i.e.} \quad \max\left(0,\, -y^{(i)}\,\mathbf{w}^\top f(\mathbf{x}^{(i)})\right)$$

Gradient for $y = +1$: $\nabla_{\mathbf{w}}\mathcal{L} = 0$ if correct, $-f(\mathbf{x})$ if wrong. SGD step $\mathbf{w} \leftarrow \mathbf{w} - \alpha\nabla\mathcal{L} = \mathbf{w} + \alpha f(\mathbf{x})$, exactly the perceptron update. Symmetric for $y = -1$: gradient $+f(\mathbf{x})$, update $\mathbf{w} - \alpha f(\mathbf{x})$.

Plotted against $z = \mathbf{w}^\top f(\mathbf{x})$ for a positive example, the loss is flat at 0 for $z \geq 0$ and a line of slope -1 for $z < 0$. It is a "hinge" at 0, convex, and piecewise linear. Lecture visualization: in weight space the total loss is 0 inside a wedge of separating weight vectors. SGD walks downhill to that flat plateau, which is convergence. Removing a point that doesn't constrain the boundary leaves the wedge unchanged.

General SGD template (lecture 1.1.2)

Objective: $\sum_{i=1}^{D} \mathcal{L}(\mathbf{x}^{(i)}, y^{(i)}, \mathbf{w})$, linear over examples. Loop: for each epoch, for i up to D: sample $j \in [1, D]$, $\mathbf{w} \leftarrow \mathbf{w} - \alpha\nabla_{\mathbf{w}}\mathcal{L}(\mathbf{x}^{(j)}, y^{(j)}, \mathbf{w})$. The gradient points toward higher loss, so subtract it. Perceptron, logistic regression and neural networks are all instances (Topic 7).

2.5 Past exam problems

Midterm Fall 2022 Q3(a) — rule $\mathbf{w}^\top\mathbf{x} > 0$

Points (x_1, x_2, y): $(1, 0, +)$, $(1, 1, +)$, $(0, 1, -)$. One pass from $\mathbf{w} = 0$. Final $\mathbf{w}$?

Solution. $(1,0)$: score $0 \Rightarrow$ pred $-$, wrong $\Rightarrow \mathbf{w} = (1,0)$. $(1,1)$: score $1 \Rightarrow +$, ok. $(0,1)$: score $0 \Rightarrow -$, ok.

Official answer: $\boxed{[1, 0]}$.

Midterm Fall 2022 Q3(b)

Add a 4th, *negative* point. Describe the region where the perceptron will *not* converge.

Solution. No bias, so the boundary passes through the origin. Feasible $\mathbf{w}$ for the 3 points (rule > 0): $w_1 > 0$ (from $(1,0)+$), $w_1 + w_2 > 0$ (from $(1,1)+$), $w_2 \leq 0$ (from $(0,1)-$). So $\mathbf{w}$ ranges from direction $(1,0)$ to just short of $(1,-1)$. A new negative $\mathbf{x}$ makes the problem infeasible iff $\mathbf{w}^\top\mathbf{x} > 0$ for *every* feasible $\mathbf{w}$. At $\mathbf{w} = (1,0)$ that needs $x_1 > 0$; as $\mathbf{w} \to (1,-1)$ it needs $x_1 - x_2 \geq 0$.

Official answer: $x_2 \leq x_1$ and $x_1 > 0$. Picture: the boundary can rotate like a clock hand from "noon" (vertical) to "1:30" (45°) with negatives on the left. Any negative point to the right of every such boundary breaks separability.

Midterm Fall 2022 Q3(c)

Add $(4, 2, -)$. Does any neural network classify all points correctly?

Official answer: Yes. Neural networks are universal function approximators, and these 4 points (no

two identical with different labels) can be separated by a nonlinear function.

Midterm Fall 2023 Q2(b) — this exam uses $\mathbf{w}^\top f(\mathbf{x}) \geq 0$ (0 is positive)

Vectors from Q2(a): [0,0,0]+, [0,1,0]+, [0,0,1]− (greatq is unknown, so only *not* fires), [1,0,1]−. With rule ≥ 0: ex1 score 0 → + ok; ex2 0 → + ok; ex3 0 → + **wrong** ⇒ $\mathbf{w} = [0, 0, -1]$; ex4 score -1 → − ok.

Official answer: $\boxed{[0, 0, -1]}$. **Contrast (lecture rule > 0):** ex1 wrong but $f(\mathbf{x}) = 0$ so no change; ex2 wrong ⇒[0,1,0]; ex3 score 0 → − ok; ex4 − ok ⇒ [0,1,0]. The tie rule changes the answer, so **always use the rule the question states**.

Midterm Fall 2023 Q1(1) (4 pts, select all)

I. Perceptron always converges. II. LR with BoW features is convex. III. SGD can optimize LR. IV. Perceptron = SGD with a certain loss. V. As epochs→ ∞, LR and perceptron give the same boundary. VI. Same, if data are separable.

Official answer: II, III, IV. VI is false: both may classify the training points identically, but the actual weight vectors and boundaries differ.

Canvas Exercise 1.1 Q1

Features [good, bad, not]; "good" +, "not good" −, "bad" −; rule > 0; run to convergence.

Solution. Epoch 1: [1,0,0], then score 1 on [1,0,1] ⇒ wrong ⇒[0,0,-1]; "bad" score 0 ok. Epoch 2: "good" score 0 wrong ⇒[1,0,-1]; others ok. Epoch 3: no mistakes.

Answer: [1,0,-1].

Traps

- **Tie rule:** lecture uses > 0 ($0 \rightarrow$ negative). Fall 2022 uses > 0; Fall 2023 uses ≥ 0. Use whatever the question says.
- A mistaken example with $f(\mathbf{x}) = \mathbf{0}$ produces no change (adding the zero vector).
- Update only on mistakes. Logistic regression updates on every example (Topic 3).
- Don't confuse "one epoch" with "until convergence".
- No bias feature means the boundary passes through the origin. That matters for separability questions like Fall 2022 Q3(b).

2.6 Practice

Practice

1. Features [great, awful, plot]. Data: "great plot" +, "awful plot" −, "plot" −, "great" +. Rule > 0, $\alpha = 1$, $\mathbf{w} = 0$. Trace to convergence.
2. Same data, $\alpha = 0.5$. What changes?
3. Same data, but start at $\mathbf{w} = [0, 0, 1]$, $\alpha = 1$. Trace one epoch.
4. Write the perceptron loss for a negative example and its gradient in both cases.
5. Are these 2-D points separable by a line through the origin: $(1, 1)+$, $(-1, -1)+$, $(1, -1)-$?

Explain.

Answers

1. Ep1: [1,0,1] score 0 $\rightarrow$ wrong $\Rightarrow$[1,0,1]; [0,1,1] score 1 wrong $\Rightarrow$[1,-1,0]; [0,0,1] score 0 ok; [1,0,0] score 1 ok. Ep2: scores 1, -1, 0, 1, all correct.

Answer: [1,-1,0].

2. Every weight is halved, [0.5,-0.5,0]. Same decisions, same mistakes.

3. "great plot" score 1 ok; "awful plot" score 1 wrong $\Rightarrow$[0,-1,0]; "plot" score 0 ok; "great" score 0 wrong $\Rightarrow$[1,-1,0]. Nonzero init changes the path.

4. $\mathcal{L} = \max(0, \mathbf{w}^\top f(\mathbf{x}))$ (with the > 0 rule, a score of 0 counts as correct). Gradient 0 if $\mathbf{w}^\top f(\mathbf{x}) \leq 0$; $+f(\mathbf{x})$ if $\mathbf{w}^\top f(\mathbf{x}) > 0$, so the update is $\mathbf{w} - \alpha f(\mathbf{x})$.

5. No. $(1,1)$ and $(-1,-1)$ are negatives of each other, so $\mathbf{w}^\top(1,1) > 0$ forces $\mathbf{w}^\top(-1,-1) < 0$. Both can't be positive.

TOPIC 3

Binary Logistic Regression

Sources: Lecture 1.3.0 Logistic Regression; slides seg-6; Exercise 1.2; A1 Part 2; Fall 2022 Q1(3); Fall 2023 Q1(1).

In plain words

Same score $z = \mathbf{w}^\top f(\mathbf{x})$ as the perceptron, but squash it through the S-shaped logistic (sigmoid) function to get a probability that the label is $+1$. Train by making the observed labels as probable as possible (maximum likelihood), which is the same as minimizing negative log-likelihood with SGD. The update looks like the perceptron's, scaled by "how wrong" the model was: $1 - P(\text{correct label})$. Confident and correct means a tiny update. Confident and wrong means essentially the perceptron update.

3.1 Model

Binary LR (lecture notation, $y \in \{-1, +1\}$)

$$P(y = +1 \mid \mathbf{x}) = \frac{e^{\mathbf{w}^\top f(\mathbf{x})}}{1 + e^{\mathbf{w}^\top f(\mathbf{x})}} = \sigma(\mathbf{w}^\top f(\mathbf{x})), \qquad P(y = -1 \mid \mathbf{x}) = \frac{1}{1 + e^{\mathbf{w}^\top f(\mathbf{x})}} = 1 - P(y = +1 \mid \mathbf{x}).$$

Logistic/sigmoid: $\sigma(z) = \dfrac{e^z}{1 + e^z} = \dfrac{1}{1 + e^{-z}}$. Properties: $\sigma(0) = 0.5$, $\sigma(z) \to 1$ as $z \to \infty$, $\to 0$ as $z \to -\infty$, $\sigma(-z) = 1 - \sigma(z)$.
Decision rule: return $+1$ iff $P(y = +1|\mathbf{x}) > 0.5 \iff \mathbf{w}^\top f(\mathbf{x}) > 0$, the same linear boundary as the perceptron. The probabilistic view only matters for *training*.

- **Discriminative** probabilistic model: models $P(y|\mathbf{x})$. A **generative** model models $P(\mathbf{x}, y)$ (Naive Bayes; HMMs in Topic 20).
- "Maximum entropy" (MaxEnt) classifier is another name for logistic regression (lecture 1.4.0).

3.2 Training objective

From likelihood to loss

1. Likelihood of the data: $\max_{\mathbf{w}} \prod_{i=1}^{D} P(y^{(i)} \mid \mathbf{x}^{(i)})$ (discriminative: condition on $\mathbf{x}$).
2. Log is monotonic, so maximize the log-likelihood: $\max_{\mathbf{w}} \sum_{i=1}^{D} \log P(y^{(i)} \mid \mathbf{x}^{(i)})$.
3. Negate to get a loss: $\min_{\mathbf{w}} \sum_{i=1}^{D} -\log P(y^{(i)} \mid \mathbf{x}^{(i)})$, the **negative log-likelihood (NLL)**.
4. Per-example loss $\mathcal{L}_i = -\log P(y^{(i)}|\mathbf{x}^{(i)})$. Optimize with SGD.

3.3 Gradient derivation (be able to reproduce)

Let $z = \mathbf{w}^\top f(\mathbf{x})$. **Case $y = +1$:**

$$\mathcal{L} = -\log \frac{e^z}{1+e^z} = -z + \log(1+e^z)$$

$$\nabla_{\mathbf{w}} \mathcal{L} = -f(\mathbf{x}) + \frac{1}{1+e^z} \cdot e^z \cdot f(\mathbf{x}) \qquad \text{(chain rule; } \nabla_{\mathbf{w}} z = f(\mathbf{x})\text{)}$$

$$= -f(\mathbf{x}) \left(1 - \frac{e^z}{1+e^z}\right) = -f(\mathbf{x}) \left(1 - P(y=+1|\mathbf{x})\right).$$

Case $y = -1$: $\mathcal{L} = -\log \frac{1}{1+e^z} = \log(1+e^z)$, so $\nabla_{\mathbf{w}} \mathcal{L} = f(\mathbf{x})\, P(y=+1|\mathbf{x}) = f(\mathbf{x}) \left(1 - P(y=-1|\mathbf{x})\right)$.

LR SGD updates (Exercise 1.2 form)

$$\begin{aligned} y=+1: &\quad \mathbf{w} \leftarrow \mathbf{w} + \alpha\, f(\mathbf{x}) \left(1 - P(y=+1|\mathbf{x})\right) \\ y=-1: &\quad \mathbf{w} \leftarrow \mathbf{w} - \alpha\, f(\mathbf{x}) \left(1 - P(y=-1|\mathbf{x})\right) \end{aligned} \iff \boxed{\mathbf{w} \leftarrow \mathbf{w} + \alpha\, y\, f(\mathbf{x}) \left(1 - P(y|\mathbf{x})\right)}$$

0/1-label form (A1 code): with $y \in \{0,1\}$ and $p = P(y=1|\mathbf{x})$: $\mathbf{w} \leftarrow \mathbf{w} + \alpha\,(y-p)\, f(\mathbf{x})$. Same update: for $y=1$, $y-p = 1-p$; for $y=0$, $y-p = -p = -(1-P(y=0|\mathbf{x}))$.

3.3.1 Relation to the perceptron

- If $P(\text{correct label}) \approx 1$: factor ≈ 0, so (almost) no update. The perceptron also skips correct examples.
- If $P(\text{correct label}) \approx 0$ (confidently wrong): factor ≈ 1, so the update is $\pm\alpha f(\mathbf{x})$, the perceptron update.
- **Exercise 1.2:** the update is numerically closest to the perceptron when $P(y{=}+1|\mathbf{x})$ or $P(y{=}-1|\mathbf{x})$ is close to 1 (that is, close to 0 or 1, not 0.5).
- As a function of z (for $y=+1$), the LR loss $\log(1+e^z) - z = \log(1+e^{-z})$ is a smooth version of the perceptron's hinge. It approaches 0 for large positive z and $\approx -z$ for large negative z. Unlike the perceptron, LR updates on *every* example, even correct ones.
- Lecture conclusion: the two behave almost identically; use whichever is convenient.

Unified loss in terms of the margin yz

Perceptron: $\max(0, -yz)$. Logistic: $\log(1+e^{-yz})$. Both are convex in $\mathbf{w}$. LR with BoW features is a **convex** optimization problem (Fall 2023 Q1(1) II true), so SGD with a suitable step size finds the global optimum.

3.4 Worked numeric example

Worked example: one LR step each for a positive and a negative example

Features (last is the bias feature): $f(\mathbf{x}) = [1, 0, 1, 1]$, $\mathbf{w} = [1, -2, 0.5, -1]$, $y = +1$, $\alpha = 0.1$.

1. $z = 1(1) + 0(-2) + 1(0.5) + 1(-1) = 0.5$.
2. $P(y{=}+1|\mathbf{x}) = \sigma(0.5) = 1/(1+e^{-0.5}) = 0.62246$.
3. Loss $= -\ln 0.62246 = 0.47408$.

4. Gradient $= -(1 - 0.62246) f(\mathbf{x}) = -0.37754\, f(\mathbf{x})$.
5. Update: $\mathbf{w} \leftarrow \mathbf{w} + 0.1(0.37754) f(\mathbf{x}) = [\,1.03775,\ -2,\ 0.53775,\ -0.96225\,]$.

Next example: $f(\mathbf{x}) = [\,0, 1, 1, 1\,]$, $y = -1$, using the new $\mathbf{w}$.

1. $z = -2 + 0.53775 - 0.96225 = -2.42449$, so $P(+1|\mathbf{x}) = \sigma(-2.42449) = 0.08132$ and $P(-1|\mathbf{x}) = 0.91868$.
2. Loss $= -\ln 0.91868 = 0.08482$ (small: already confident and correct).
3. Update: $\mathbf{w} \leftarrow \mathbf{w} - 0.1(1 - 0.91868) f(\mathbf{x}) = \mathbf{w} - 0.008132\, f(\mathbf{x}) = [\,1.03775,\ -2.00813,\ 0.52962,\ -0.97038\,]$.

Only the features that fire ($f_j \neq 0$) change.

Calculator recipe: sigmoid, log-loss, update

1. Compute $z = \sum_j w_j f_j$ (only nonzero f_j).
2. $p = 1 \div (1 + e^{-z})$: key `e^(-z)`, add 1, take reciprocal.
3. Loss for $y = +1$: $-\ln p$; for $y = -1$: $-\ln(1 - p)$. Use `ln` (natural log) unless told otherwise.
4. Scale factor $s = \alpha(1 - p)$ for $y = +1$, or $-\alpha p$ for $y = -1$. New $w_j = w_j + s\, f_j$.
5. Keep 5+ significant digits until the final step, then round as instructed.

3.5 Past exam / exercise problems

Midterm Fall 2022 Q1(3)

A graph of an S-curve from 0 to 1.

Official answer: B: a sigmoid; as $x \to -\infty$ it goes to 0, as $x \to +\infty$ it goes to 1. Distractors: tanh ranges over $(-1, 1)$, not $(0, 1)$; a curve going $1 \to 0$ would be $\sigma(-x)$.

Canvas Exercise 1.2

When is the LR update most similar to the perceptron's?

Answer: When $P(y{=}-1|\mathbf{x}) \approx 1$ or $P(y{=}+1|\mathbf{x}) \approx 1$. The factor $(1 - P)$ is then ≈ 0 or ≈ 1. Exactly 0 or 1 recovers "$\pm\alpha f(\mathbf{x})$ on mistakes, nothing otherwise".

Traps

- The gradient sign: the SGD step *subtracts* the gradient. For $y = +1$ the gradient is $-f(\mathbf{x})(1 - p)$, so you *add* $\alpha f(\mathbf{x})(1 - p)$.
- For $y = -1$ use $1 - P(y{=}-1|\mathbf{x}) = P(y{=}+1|\mathbf{x})$. Don't use $1 - P(+1)$.
- The LR decision threshold is 0.5 on the probability, i.e. 0 on the score. Changing the decision threshold doesn't change training.
- Logistic regression is linear: it cannot solve XOR/"not good vs. not bad" with unigram features, just like the perceptron.
- Use natural log unless told otherwise; NLL in PyTorch uses ln.

3.6 Practice

Practice

1. $\mathbf{w} = [2, -1]$, $f(\mathbf{x}) = [1, 3]$. Compute $P(y{=}+1|\mathbf{x})$, the predicted label, and the NLL if the true $y = -1$.
2. With $\alpha = 0.5$, give the updated $\mathbf{w}$ for Q1.
3. Show that $P(y|\mathbf{x}) = \sigma(y\,\mathbf{w}^\top f(\mathbf{x}))$ for $y \in \{-1, +1\}$.
4. An example has $P(\text{correct}) = 0.99$. Roughly what fraction of the perceptron-sized step does LR take?
5. Why does LR still update on correctly classified examples, and why can that be useful?

Answers

1. $z = 2 - 3 = -1$, $p = \sigma(-1) = 0.26894$, predict -1; NLL $= -\ln(1 - 0.26894) = -\ln 0.73106 = 0.31326$.
2. $\mathbf{w} \leftarrow \mathbf{w} - 0.5(0.26894)[1, 3] = [1.86553,\ -1.40341]$.
3. $y = +1$: $\sigma(z)$. $y = -1$: $\sigma(-z) = 1/(1 + e^z) = P(y{=}-1|\mathbf{x})$.
4. $1 - 0.99 = 0.01$, about 1%.
5. The loss is never exactly 0, so the gradient is nonzero. It keeps pushing toward larger margins, i.e. more confident correct predictions.

TOPIC 4

Sentiment Analysis: Useful Features

Sources: Lectures 1.1.1 Sentiment Analysis & Basic Feature Extraction, 1.4.0 Sentiment Analysis; slides seg-2, seg-7; Exercise 1.0; A1 Part 3; Fall 2022 Q2(c); Fall 2023 Q2(e), Q1(5–6).

In plain words

Sentiment is mostly carried by *content words* (great, awful, enjoyable) and short phrases (*watch again*, *not good*). Unigram bag-of-words captures the words, and bigrams capture short phrases and local negation. What BoW misses is structure: negation that scopes over several words, and discourse markers like "but" that override what came before. Simple BoW classifiers were surprisingly strong for years. Large pre-trained models (BERT etc.) now dominate.

4.1 What makes sentiment hard (lecture examples)

example	phenomenon
the movie was great! would watch again! (+)	polar word *great*; phrase *watch again*
the film was awful; I'll never watch again (−)	*watch again* appears in a negative example: negation (*never*)
the movie was gross and overwrought, but I liked it (+)	discourse: *but* overrides earlier negative words
this movie was not really very enjoyable (−)	negation scopes over several words; unigram *enjoyable* misleads

4.2 Useful features (what the exam can ask)

1. **Unigram indicators/counts** of sentiment words. Strong baseline.
2. **Bigrams** (and n-grams): *not good*, *watch again*, *not bad*. They capture local negation and phrases.
3. **Negation marking:** after a *not*, extract a "not_X" feature for every following word X: *not really*, *not very*, *not enjoyable*. Without a syntactic parser you can't determine the negation scope reliably.
4. **Presence vs. frequency:** the 2002 study (Pang, Lee & Vaithyanathan) compared both. Presence (0/1) is a common choice for sentiment.
5. **Preprocessing:** lowercasing and stop-word removal often help sentiment. *Keep negation words*: the A1 `BetterFeatureExtractor` removes spaCy stop words *except* {no, nor, not, never}.
6. Others suggested in A1: tf-idf weighting, clipping word frequencies, discarding rare words, first-word features, other n-grams.
7. **Task-specific indicator features**, e.g. "count of *not* is odd" (Exercise 1.0).

4.3 History (lecture 1.4.0) — MC fodder

- **Pang, Lee & Vaithyanathan (2002)**: simple feature sets do pretty well. Best were unigram and unigram+bigram features with Naive Bayes, MaxEnt (=LR) and SVMs.
- **Wang & Manning (2012)**: revisited BoW. A modified Naive Bayes (NB-SVM) matched or beat a 2012 neural recursive autoencoder.
- **Kim (2014)**: CNNs beat these substantially.
- Today: large pre-trained networks (BERT, XLNet, . . .) on SST binary classification, going from about 90 (BiLSTMs) to about 97.

4.4 A1 connection

Binary SST sentences (neutral removed); labels 0/1. Required dev accuracy: unigram perceptron $\geq 74\%$, unigram LR $\geq 77\%$. Q3: bigram indicators on adjacent word pairs. Q4: a "better" extractor that is more than just unigrams+bigrams. Always-positive baseline: 444/872 = 50.9% dev.

4.5 Past exam problems

Midterm Fall 2022 Q2(c)

Your unigram BoW model has been in production for years. A new, very negative slang word appears that the model has never seen. Name two ways to update the model.

Official answer: (1) Add labeled sentences containing the word and retrain (or continue training) for a few epochs. (2) Directly add a feature for the word with a large negative weight.

Midterm Fall 2023 Q2(e)

Typos can be anywhere in a word. Which is more effective for robustness: subword tokenization or explicit typo repair (closest vocabulary word by edit distance)?

Official answer: Typo repair is generally less fragile. Subword tokenization breaks typo'd words into odd pieces (`greqat`→ gre q at), while edit-distance repair maps `goodx`→`good`.

Midterm Fall 2023 Q1(5)–(6) (preview of Topic 11, Topic 12)

A unidirectional RNN reads "The play was terrible . . . lackluster . . . unconvincing . . . At least the dinner was excellent." and predicts very positive. Why?

Official answer: D: an RNN's state is heavily influenced by recent tokens. Q6 (what might work better, select all):

Official answer: I (BoW: more negative words overall), II (DAN), III (Transformer). All are order-insensitive or attend globally.

Traps

- Removing stop words can delete *not/no/never* unless you exclude them. That destroys negation features.
- Bigrams increase the feature space. They help negation only locally (*not really very enjoyable* has no *not enjoyable* bigram).

- Both "Indicator(contains not)" and "Count(not)" are only partially useful for alternating negation. The right feature depends on the data pattern (Exercise 1.0).

4.6 Practice

Practice

1. List the features extracted by the "not_X" trick for "it was not very funny at all".
2. Design one feature that lets a linear model handle "X, but Y" sentences where Y decides the label.
3. Why might presence features beat counts for short sentiment snippets?

Answers

1. not_very, not_funny, not_at, not_all (every word after *not*; you might stop at punctuation).
2. e.g. prefix every word after *but* with `AFTERBUT_` (or weight post-*but* words more), so the model learns separate weights for clause-final sentiment.
3. Repeating a word rarely makes the sentiment stronger in proportion. Counts let frequent but neutral repetitions dominate the score.

TOPIC 7

Optimization: SGD, Step Size, Initialization

Sources: Lectures 1.1.2 Basics of Learning, Gradient Descent, 1.4.1 Optimization Basics, 2.2.1 Neural Net Implementation, 2.2.2 Neural Net Training, Optimization; slides seg-3, seg-8, seg-15, seg-16; Exercises 2.3a, 2.3b; A2 Part 1 (Q1); Fall 2023 Q1(1).

In plain words

Training means searching for weights that make the total loss small. SGD repeatedly picks one example (or a batch), computes which direction makes the loss go *up* (the gradient), and takes a step of size α the other way. Too small a step is slow. Too large a step overshoots, oscillates or diverges. For linear models the loss is convex, so where you start doesn't matter. For neural networks it's non-convex, so the starting point (initialization) matters a lot. In particular, all-zero initialization breaks learning.

7.1 The optimization problem

Objective and SGD (lecture)

$$\min_{\mathbf{w}} \ \mathcal{L}(\mathbf{w}) = \sum_{i=1}^{D} \mathcal{L}(\mathbf{x}^{(i)}, y^{(i)}, \mathbf{w}) \qquad \text{(training data fixed; } \mathbf{w} \text{ varies)}$$

SGD: repeatedly pick an example i and set $\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{x}^{(i)}, y^{(i)}, \mathbf{w})$.
The objective is a *sum over examples* (linear over examples). That's why one example's gradient is a cheap, noisy estimate of the full gradient.

- **Batch/full gradient descent:** sum the gradient over all D examples, then take one step. **Minibatch SGD:** average or sum over a batch (Topic 9).
- **Instances:** perceptron (perceptron loss), logistic regression (NLL), neural networks (NLL through backprop).
- **Convexity:** perceptron loss and LR's NLL are convex in $\mathbf{w}$ (with fixed features). Neural network objectives are **non-convex**.

7.2 Step size: the quadratic example

Worked example: lecture 1.4.1: $\mathcal{L}(w) = w^2$, start at $w = -1$

Gradient $\nabla \mathcal{L} = 2w$, so the update is $w \leftarrow w - \alpha \cdot 2w = (1 - 2\alpha)\, w$.

α	sequence $w_0, w_1, \ldots$	behaviour
0.1	$-1, -0.8, -0.64, -0.512, \ldots$	slow, monotone convergence
0.25	$-1, -0.5, -0.25, -0.125, \ldots$	faster
0.5	$-1,\ \mathbf{0}, 0, 0$	**one step to the optimum**
0.75	$-1, 0.5, -0.25, 0.125, \ldots$	oscillating, still converging
1	$-1, 1, -1, 1, \ldots$	**oscillates forever** (lecture)
1.1	$-1, 1.2, -1.44, 1.728, \ldots$	diverges

General rule for this function (derived): converges iff $|1 - 2\alpha| < 1 \iff 0 < \alpha < 1$; monotone for $\alpha < \frac{1}{2}$; exact in one step at $\alpha = \frac{1}{2}$.

Worked example: Assignment 2 Q1: $f(x_1, x_2) = (x_1 - 1)^2 + 8(x_2 - 1)^2$, start at the origin

1. Gradient: $\nabla f = \big(2(x_1 - 1),\ 16(x_2 - 1)\big)$. Optimum $(1, 1)$.
2. Per-coordinate error after one step: $(x_1 - 1) \leftarrow (1 - 2\alpha)(x_1 - 1)$ and $(x_2 - 1) \leftarrow (1 - 16\alpha)(x_2 - 1)$.
3. Convergence needs *both* $|1 - 2\alpha| < 1$ and $|1 - 16\alpha| < 1$, so $0 < \alpha < \frac{1}{8}$. The steep coordinate (x_2) sets the tipping point: at $\alpha = 1/8$, x_2 oscillates; above it, SGD diverges (e.g. `-lr 1`).
4. Distance after k steps: $\sqrt{(1 - 2\alpha)^{2k} + (1 - 16\alpha)^{2k}}$. One choice is $\alpha \approx 0.1066$, which reaches distance $0.0959 < 0.1$ in **10** steps. No constant α does it in 9 (the best possible after 9 steps is 0.1233).
5. Lesson: **one global step size has to suit every direction**. A flat direction wants a large step and a steep direction a small one. This motivates adaptive methods and Newton's method.

7.3 Choosing step sizes (lecture recipe)

Practical step-size choices

1. **Try them out** over orders of magnitude: $1, 10^{-1}, 10^{-2}, \ldots$ Neural models often need small ones (10^{-4}, 10^{-5}).
2. **Large → small schedules:** $\alpha_t \propto 1/t$ or $1/\sqrt{t}$ for epoch t (fixed schedule).
3. **Decrease on plateau:** reduce α when held-out (dev) performance stagnates.

Second-order and adaptive methods

Newton's method: $\mathbf{w} \leftarrow \mathbf{w} - \left(\nabla^2_{\mathbf{w}} \mathcal{L}\right)^{-1} \nabla_{\mathbf{w}} \mathcal{L}$, using the inverse Hessian (curvature). On a quadratic it jumps straight to the optimum. But the Hessian is **quadratic in the number of features**, so it's infeasible for big models.

Adagrad, Adadelta, Adam: "adaptive" methods, cheap *approximations* to the inverse Hessian that are **linear** in the number of parameters. They rescale the step per coordinate. A rare feature (someone's name) and a common one (*said*) shouldn't share one step size.

Adam (Kingma & Ba 2015) = adaptive per-coordinate step size + **momentum** (an exponentially weighted average of past gradients). It is the default for NLP in this course. Wilson et al. (NeurIPS 2017): adaptive methods can fit training data well but generalize worse at test time than carefully tuned SGD.

7.4 Regularization (lecture: "don't really use")

Classical statistics says don't fully optimize the training loss (bias–variance tradeoff), and add explicit regularization. In this course we usually don't add a regularizer. Instead we rely on **early stopping**, **imperfect optimization**, and tricks like **dropout**.

7.5 Initialization (neural networks)

Why initialization matters

1. **Can't use zeros for parameters that produce hidden layers.** With $\mathbf{z} = g(Vf(\mathbf{x}))$ and $V = 0$, $\mathbf{z} = g(\mathbf{0}) = \mathbf{0}$ for tanh and ReLU. The output layer sees all zeros, every hidden value and gradient is 0, and nothing ever changes. Nonzero random values also **break symmetry** between hidden units.
2. **Don't initialize too large:** tanh/sigmoid units **saturate**. Inputs like ± 5 sit on the flat part of the curve, so the gradient passed back is tiny and learning stalls. ReLU has an analogous problem: if a unit's input is always negative, it outputs 0 with gradient 0 (it's "stuck").
3. **Glorot/Xavier initializer:** $U\left[-\sqrt{\frac{6}{\text{fan-in}+\text{fan-out}}}, +\sqrt{\frac{6}{\text{fan-in}+\text{fan-out}}}\right]$ (fan-in/fan-out = number of inputs/outputs of the layer). Goal: keep the variance of activations and gradients about the same across layers, so one step size works everywhere.
4. **Batch normalization** (Ioffe & Szegedy 2015): during training, shift and rescale each layer's activations to mean 0, variance 1 over the batch. Useful for very deep nets, mostly in vision.

Linear models vs. NNs: zero initialization is fine for perceptron/LR (convex, every weight gets its own gradient from the features). For NNs it's fatal, and different random initializations can converge to different solutions.

Canvas Exercise 2.3a

Q1: Network $\mathbf{y} = \text{softmax}(Wg(Vf(\mathbf{x})))$ with all layers initialized to zero.

Answer: It won't learn; the gradients are all 0. Q2: What can improve performance or learning speed?

Answer: Adam (momentum-based) instead of SGD; Xavier initialization; batching. *Not* very large random initial values (saturation).

Canvas Exercise 2.3b (XOR with the PyTorch FFNN example)

Q1: Run 5–10 times on the XOR data.

Answer: Sometimes fits, inconsistent across random initializations (non-convex). Q2: Remove the example $[1, 1]$.

Answer: Fits perfectly (the remaining 3 points are linearly separable, an easy problem). Q3: Zero initialization instead of Xavier.

Answer: Always fails to fit.

Midterm Fall 2023 Q1(1), revisited

"LR with BoW features is convex" is true. "SGD can optimize LR" is true. "The perceptron is SGD on some loss" is true. Neural networks are non-convex, which is why initialization matters for them but not for LR.

Calculator recipe: gradient-descent steps on a quadratic

For $f(\mathbf{x}) = \sum_j a_j(x_j - c_j)^2$: error in coordinate j multiplies by $(1 - 2a_j\alpha)$ each step. After k steps the error is $(1 - 2a_j\alpha)^k\times$ the initial error. Converges iff $\alpha < 1/a_{\max}$.

Traps

- The SGD update *subtracts* the gradient (it minimizes loss). If a question maximizes log-likelihood, gradient *ascent* adds it. Same step.
- "Oscillates" (stays bounded, $|1 - 2a\alpha| = 1$) is different from "diverges" ($|1 - 2a\alpha| > 1$).
- Zero initialization: fatal for hidden layers, harmless for linear models. Large initialization: saturation.
- Newton's method is quadratic in the number of features; Adam etc. are linear.
- With α constant and $\mathbf{w}_0 = 0$, the perceptron is α-invariant. LR and NNs are not.

7.6 Practice

Practice

1. $\mathcal{L}(w) = 3w^2$, $w_0 = 2$. Give w_1, w_2 for $\alpha = 0.1$. Which α reaches 0 in one step? For which α does it diverge?
2. $f(x_1, x_2) = x_1^2 + 4x_2^2$, start $(1, 1)$, $\alpha = 0.2$. Two steps of GD. Converging?
3. A hidden layer uses ReLU and every weight is initialized to -10 with nonnegative BoW inputs. What happens?
4. Name the two ideas Adam combines. Which method uses the inverse Hessian, and why isn't it used for big networks?

Answers

1. $w \leftarrow (1 - 6\alpha)w$: $w_1 = 0.8$, $w_2 = 0.32$. One step at $\alpha = 1/6$. Diverges for $\alpha > 1/3$ (oscillates forever at exactly $1/3$).
2. $x_1 \leftarrow (1 - 0.4)x_1 = 0.6x_1$, $x_2 \leftarrow (1 - 1.6)x_2 = -0.6x_2$: $(0.6, -0.6)$, then $(0.36, 0.36)$. Converging, with x_2 oscillating in sign.
3. Every pre-activation is ≤ 0, so ReLU outputs 0 with zero gradient. The units are dead and nothing learns through them.
4. Adaptive per-coordinate step size + momentum. Newton's method; the Hessian is quadratic in the number of parameters.